
What Happens to a Monitor’s Accuracy When You Train Against It

Abraham Yeung*
Stanford University

Anagha Ramaswamy
Stanford University

Abstract

A linear probe on a language model’s hidden states predicts whether the model’s next answer will be correct. On held-out prompts it ranks a correct answer above an incorrect one 98% of the time. The probe we train against is a second fit of the same kind, on the answers that reinforcement learning samples. Used as the reward, it lowers true accuracy by 31 percentage points.

The reward probe’s own accuracy does not show this for 30 to 40 training steps. Scored at every checkpoint on that checkpoint’s fresh answers, its AUROC first changes at step 40, or at step 50 if each answer is graded by its last answer block rather than its first, although task accuracy is already significantly lower at step 10. An LLM judge that is verifiably a fixed function, and was never trained against, ends the run with the AUROC it started with while accuracy falls from 54% to 23%. A monitor’s accuracy statistics describe the monitor together with the population it scores, and deploying the monitor is what moves the population. A stable AUROC says the monitor still separates correct answers from incorrect ones. It does not say how many answers are correct. The share of answers above a threshold frozen at the start moves without labels, by step 10 for the probe and by step 30 for the judge. For the probe that movement is partly guaranteed by the objective, and neither has been calibrated against matched benign training.

Held-out accuracy also did not rank our rewards by how safe they were to train against. Of two rewards trained under one recipe, a monitor built only from 39 features of the answer text reads higher as a held-out monitor than a probe on activations with those features removed (0.925 against 0.834). As a reward it drives accuracy to exactly zero, where the probe left 0.168. Nothing in the objective rewards evading the monitor. The policy is doing what it was asked.

1 Introduction

A probe is a small classifier trained on a model’s internal activations to predict some property of the model’s behavior. Probes are increasingly proposed as control signals rather than as analysis tools, for example refusal directions (Arditi et al., 2024), correctness probes (Burns et al., 2023) and deception probes (Pacchiardi et al., 2024). Placing a probe inside the training loop and optimizing against it is less well understood than evaluating the probe from outside the loop.

Reward overoptimization (Gao et al., 2023; Coste et al., 2024; Skalse et al., 2022; Denison et al., 2024) shows that a proxy reward is gamed once a policy can optimize against it, with no adversarial intent required. Monitor evasion (Baker et al., 2025; Gupta and Jenner, 2025; Taufeeque et al., 2026; Bailey et al., 2024) shows that monitors of internal states can be evaded when the monitor enters training as a penalty. Work on LLM evaluators reaches a related conclusion from the measurement side. Agreement statistics overstate judge reliability (Zheng et al., 2023; Norman et al., 2026; Weng

*Correspondence: ayeung16@stanford.edu. Code and the regeneration script: <https://github.com/Abraham-y/reader-writer-probe-rl>.

<i>(a) the four monitors</i>				
name	what it reads	what it was fit to	AUROC	how it is used here
trace-final probe	L16 </think>	headline answers, first block	0.982	read-only; never a reward
reward probe	L16 </think>, <i>another</i> fit	$T=1$ answers, last block	0.810 / 0.945	the RL reward; scored in Fig. 1
surface model	39 text features, no activations	headline answers, first block	0.926 / 0.925	free baseline; arm B's reward
LLM judge	Qwen2.5-7B-Instruct	not fit; a fixed prompt	0.785	bystander; never optimised against
<i>(b) the four evaluation protocols, all on the same 406 prompts, at most 1,024 new tokens</i>				
name	answers per prompt	sampling	block graded	where
headline	16	$T=0.6$, top- p 0.95, top- k 20	first	§3; §4 accuracies
arm	8	$T=1$, top- p 1	first	arms A, B and C (Table 2)
probe ladder	8, fresh per checkpoint	$T=1$, top- p 1	both	§4.1, Fig. 1, Tables 3, 4
judge ladder	8, fresh per checkpoint	$T=1$, top- p 1	first	§4.2, Table 5

Table 1: Who is who. **(a)** Where two AUROCs are given they are two fits of the same features under different label rules and splits, and are *not* interchangeable: the reward probe reads 0.810 on its own protocol (last block, on answers to 300 prompts from the RL training pool, none of them among the 406) and 0.945 on the population the reward arms share (first block); the surface model reads 0.926 under the GroupKFold of §3 and 0.925 as the shipped arm-B artifact. Only the AUROC column of Table 2 is one common population, and it is the only one we rank. **(b)** “First block” grades the first <answer> block in an answer, “last block” the last. During RL, sampling stopped at the first </answer>, so the verifier only ever graded the first block; every evaluation here samples without that stop, which is why later blocks exist at all. The two ladders are independent draws from the same checkpoints, and their first-block accuracies agree to within 0.016 at every checkpoint. The headline protocol’s settings are the sampler’s defaults; its output files do not record them.

et al., 2026), and reward-model benchmark scores are weak predictors of downstream outcomes (Yang et al., 2026; Frick et al., 2024; Malik et al., 2025).

Both mechanisms are established. Less is known about what a monitor’s own quality statistics do while the policy it scores degrades. Baker et al. (2025) report that a chain-of-thought monitor’s recall falls to near zero once the policy is optimised against it, measured against regex ground truth for two specific hacks. They note that without such ground truth, one could not tell that the agent had learned to hide. That case is evasion, because the monitor entered training as a penalty. In our setting the monitor enters as a positive reward, so there is nothing to evade, and Countdown provides exact ground truth for every answer at every checkpoint. This makes it possible to measure what a monitor’s discrimination does while the policy is complying with it. In our run, the monitor’s discrimination statistics are the last thing to move. This is consistent with Beigi et al. (2026), who show that probing a policy’s internals forecasts reward hacking before the visible hack rate rises. The information exists, but it is not in the monitor’s accuracy against its target.

This applies to probes in particular. Statistics such as AUROC and precision are not properties of a classifier alone. They are properties of a classifier together with a population, and the field reports probe AUROC as if it belonged to the probe. Belinkov (2022) argues that probe accuracy does not license the claim that the model uses the probed direction. Probe accuracy also does not license a claim about the probe, because deploying the artifact changes the population on which the number was measured. The judge in §4.2 is a deterministic function that does not change. Its AUROC ends where it began, so it still discriminates, and the policy under it loses most of its accuracy.

We work with Qwen2.5-0.5B trained on Countdown arithmetic (Gandhi et al., 2024) with RLOO (Ahmadian et al., 2024). The task has an exact verifier, so a learned scorer’s divergence from ground truth can be measured directly, without confounds from preference modeling. Four different scorers are referred to in this paper as “the monitor”, and the same checkpoints are sampled and graded under four protocols. Table 1 names each, and every figure and table below states which it uses.

Contributions.

1. Held-out quality did not rank our monitors by how safe they were to train against. Our best monitor, the trace-final probe, reaches AUROC 0.982, but it is not the probe we trained against. The reward had to be computed on the answers RL samples, so a second probe was fit on them. It shares a direction with the first at cosine 0.192, and it costs 31 pp when made the reward. The controlled comparison is between two rewards trained under one recipe and scored on one population. A monitor built only from text features reads higher than a probe on activations with those features removed (0.925 against 0.834). Training against it drives accuracy to exactly

0.0000, where the probe left 0.168 (a difference of 16.8 pp, CI [14.7, 19.0]). Two single-seed runs give one counterexample, not a trend. (§4, §4.3)

2. A monitor’s discrimination statistics did not detect the decline. Task accuracy on the probe-as-reward run is significantly lower by step 10. The reward probe’s AUROC first changes at step 40 when answers are graded by their first block, the block training graded, and at step 50 when graded by the last block, the label the probe was fit to. The central case is an LLM judge that is verifiably a fixed function and was never optimised against. Over the run its AUROC ends where it began, and its likelihood ratio, which does not move with the share of correct answers, does not degrade. The judge still discriminates. Discrimination is a joint property of a classifier and a population, deploying the classifier is what moves the population, and a flat AUROC is compatible with a failing policy. Part of the probe’s flatness is mechanical. The mean score inflates while the ranking it induces stays intact, so AUROC and other rank-based statistics cannot register the change. The share of answers above a threshold frozen at step 0 registers the inflation at step 10, 30 to 40 steps before the AUROC moves, without labels. For the probe that early movement is partly guaranteed by the objective, and using the statistic as an alarm would require a matched benign control, which has not been run. (§4.1, §4.2)

2 Setup

We use Qwen2.5-0.5B on Countdown. Given 3–4 integers and a target, the model must produce an equation using each integer exactly once, checked by an exact rule-based verifier. C_{SFT} is a supervised checkpoint. C_{outcome} is 100 RLOO steps against the verifier, which raises first-block accuracy on our 406 evaluation prompts from 0.290 to 0.550. The evaluation set is 406 procedurally generated problems with no overlap against the RLOO training pool. AUROC here is the probability that a randomly chosen correct answer scores above a randomly chosen incorrect one, so 0.5 is chance. It is seed-dependent at the ~ 0.015 level, and we do not rely on a third decimal. Appendix C gives the probes, hyperparameters and estimator.

3 The probe is a good detector

A trace-final probe on layer-16 `</think>` states reaches AUROC 0.982 on C_{outcome} ’s headline answers, held out by prompt, meaning it is scored on prompts it was not fit on. As a best-of-16 selector it captures 88.8% of the accuracy gain available to any selector (CI [78.9%, 96.5%]). Free baselines come close, which matters in §4.3. `</think>` position alone scores 0.915, and a 39-feature surface model scores 0.926. As a selector the probe beats both free baselines by +3.20 pp of selection accuracy. The probe is a good detector that partly relies on a cheap surface correlate, and it is not only a length detector. Appendix D gives the baselines and the length controls.

The probe evaluated here is not the probe trained against below, and the two share little direction (cosine 0.192). A reward must be computed on the answers the policy samples during RL, at temperature 1, and the trace-final probe was fit on temperature-0.6 answers, so the reward probe is a separate fit, on temperature-1 answers to 300 prompts from the RL training pool. None of those prompts is among the 406 evaluation prompts. Its labels grade the last answer block of answers sampled without the stop at `</answer>` that RL training used, and its held-out AUROC on that protocol is 0.810. On C_{outcome} ’s headline answers graded by the first block it reads 0.951. The reward probe is also the probe scored at every checkpoint in §4.1, under both labels.

4 Using the probe as the reward destroys the task

The reward probe is now the quantity the model is trained to maximize. The reward is its $P(\text{correct})$ at `</think>`, read from the current policy’s hidden state; the probe is fixed, and the copy of the policy it reads is reloaded each round. The verifier still runs, but only to log true accuracy. It does not enter the gradient. Training runs 100 RLOO steps with the vanilla run’s learning rate, KL and entropy coefficients and sampling, but with group size 8 rather than 16, weight decay 0.01 rather than 10^{-4} , and gradient clipping at 1.0 rather than none (Appendix C).

We ran this from two starting points, since initialisation mattered more than any other factor we varied.

runA Starts from C_{outcome} , so the probe is in-distribution at step 0.

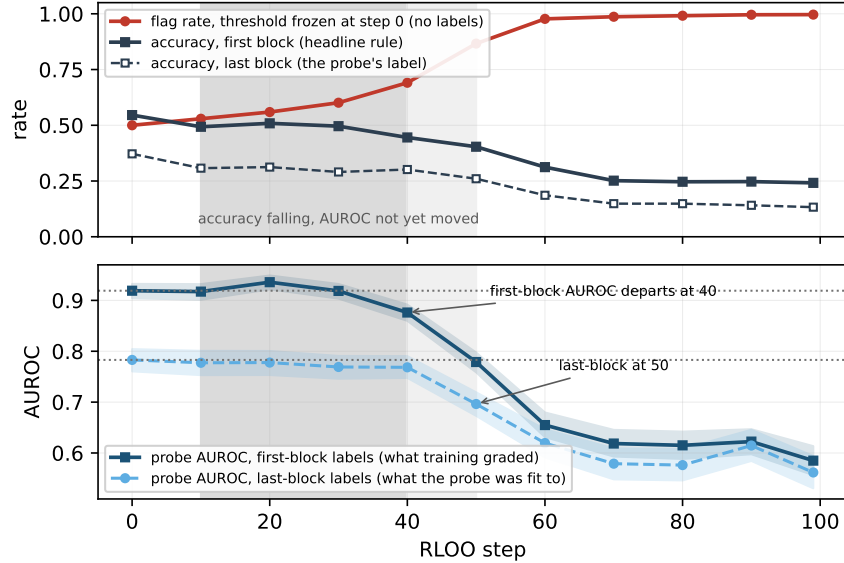


Figure 1: The monitor’s AUROC does not register the decline for 30 to 40 steps. runA, initialised from C_{outcome} , under the probe-ladder protocol: at each checkpoint, 8 fresh answers per prompt on the 406 prompts, sampled at temperature 1. **Top:** accuracy under both grading rules is significantly lower by step 10, while the fraction of answers above a threshold frozen at step 0 rises. The shaded span runs from the first significant fall in accuracy to the first significant change in AUROC, at step 40 (dark) under first-block labels and step 50 (light) under last-block labels. **Bottom:** the frozen reward probe’s AUROC under each label, with 95% prompt-clustered intervals. Both panels are computed from the same data as Tables 3 and 4.

runB Starts from C_{SFT} , so the probe is out-of-distribution at step 0.

Both probe-reward runs lose most of their accuracy. On the 406 evaluation prompts, under the headline protocol (Table 1b), runA falls to 0.236. That is -31 pp from C_{outcome} ’s 0.550. runB falls to 0.073, which is -22 pp from C_{SFT} ’s 0.290. Both runs reached a probe score of about 0.99.

The trained policy writes answers that have the form of reasoning without its content. A transparent seven-marker template score rises $1.81 \rightarrow 4.32$ between C_{outcome} and runB under the headline protocol, and its correlation with first-block correctness falls from $r=+0.643$ to $+0.023$. The probe continues to score these answers as correct. §4.3 gives a verbatim example.

4.1 The probe’s AUROC does not move for 30 to 40 steps after accuracy starts to fall

The probe’s own accuracy is scored at every checkpoint to determine when it registers that the policy trained against it is getting worse at the task.

This can be measured precisely here because the monitor is a correctness probe on a verifiable task. The reward-model literature cannot do this, since a reward model would require fresh human preference labels at every checkpoint, whereas we have the verifier. We take all eleven checkpoints of runA. At each checkpoint we sample 8 fresh answers per prompt at temperature 1 from that checkpoint (the probe ladder of Table 1b) and score them with the frozen reward probe. We compare the scores against two labels: the first block, which is what training graded, and the last block, which is what the probe was fit to predict. The probe is the only element held fixed across checkpoints. The answers it scores differ at every checkpoint because the policy has changed. Figure 1 shows the result, and Tables 3 and 4 in Appendix E give every checkpoint.

Task accuracy falls first. By step 10, first-block accuracy is 5.2 points below its step-0 value of 0.546 (95% CI $[-6.9, -3.7]$, paired over prompts), and last-block accuracy is 6.4 points below its 0.372 ($[-8.2, -4.6]$). By step 40 the falls are 10.0 and 7.0 points. Under first-block labels the probe’s AUROC starts at 0.919, rises slightly at step 20 ($+0.017$, $p = 0.022$), and first falls at step 40, by -0.043 ($p < 0.001$). Under last-block labels it starts at 0.783, does not differ detectably from step 0

through step 40 (all $p \geq 0.22$), and first moves at step 50, by -0.086 ($p < 0.001$). Either way the AUROC holds for 30 to 40 steps after accuracy has fallen, and this ordering needs no argument about the probe, since accuracy is measured by the verifier. The two labels differ because answers sampled without training’s stop at `</answer>` run on into further answer blocks the policy would never have written in training, and those later blocks are more often wrong. The judge ladder of §4.2 grades the first block, which is why its accuracy starts at 0.544 rather than 0.372.

A flat AUROC also means what it says. Until it moves, the probe still ranks the policy’s correct answers above its incorrect ones about as well as it did at step 0. What AUROC cannot report is that there are fewer correct answers to rank. Part of the reason it is flat is the metric rather than the policy. AUROC depends only on the order in which the probe ranks answers. Between steps 0 and 40 the mean probe score inflates $0.473 \rightarrow 0.677$ while that order changes little. This is close to a pure recalibration, meaning the scores shift but their order does not, and a rank-based statistic cannot register a recalibration of any size. The flatness before the AUROC moves follows from what AUROC measures and is not specific to the probe. The obvious replacements do not help. Precision at a frozen threshold is confounded with the falling share of correct answers. The prevalence-invariant LR+ falls $2.41 \rightarrow 1.65$, a larger relative drop than raw precision. Appendix B works through these.

Statistics that need no labels can register the inflation. Each of the statistics above requires correctness labels, with which the accuracy drop could be measured directly. Score saturation is one label-free statistic: the share of answers above 0.95 rises from 5.2% at step 0 to 38.8% by step 40, while on the benign ladder of Appendix B the same probe keeps it below 9% at every checkpoint. The flag rate is a finer version. We freeze a threshold at step 0 and record the share of answers that clear it. Under attack this share rises monotonically (+6%, +12%, +20%, +38% at steps 10 to 40, and eventually +99%) and does not return. Every checkpoint is scored on the same 406 prompts, so the appropriate test is paired, comparing each prompt with itself across checkpoints. The flag rate moves at the first checkpoint after training begins, by $+0.030$ [$+0.012, +0.048$] at step 10 ($p = 0.001$; one bootstrap draw of 2,000 crossed zero), whereas the AUROC first moves at step 40 or 50. The two statistics, computed from the same probe on the same answers, are therefore separated by 30 to 40 steps, and the flag-rate departure grows to $+0.496$ by step 99.

Some of that separation is expected from how the two statistics are defined. The flag rate is the threshold-crossing rate of the probe score, and the probe score is the training objective. Gradient ascent raises it by construction, so an early departure is partly guaranteed. What is not guaranteed is the size of the gap between the two, since nothing in the objective requires that the score inflate for 30 to 40 steps while the ranking it induces stays intact. The ordering between accuracy and AUROC does not share this circularity, since accuracy comes from the verifier. For a label-free signal, the non-circular case is the judge in §4.2, which the policy was never trained against and whose flag rate also moves: nominally at step 20 ($p = 0.013$), and from step 30 after a Holm correction over the judge’s 20 contrasts.

A paired test against step 0 shows that the statistic moved, not that it moved more than benign training would move it. Turning the ordering into a detector requires a benign band from a matched ladder of ordinary verifier-reward checkpoints, and our ladder has four defects, listed in Appendix B. On the prompts all its checkpoints share, benign training moves the flag rate by up to +18.3%. The attacked run first exceeds that at step 30 (+20.2%), a lead of 10 steps over the first-block AUROC and 20 over the last-block one. The flag rate’s lead therefore establishes the ordering and not a calibrated false-alarm rate. The metric argument does not depend on the benign band, and the ordering is measured on the attacked run alone. A matched control ladder would be needed to calibrate the flag rate as a detector.

Monitoring a score distribution against a frozen reference is standard practice. Li (2026) provides a better instrument for it, a sequential test that stays valid however often it is checked, run against a fixed set of human-labelled anchors, which also attributes drift to the system or to the judge. The measurement here is how much earlier, in RLOO steps, such an instrument registers the change than the monitor’s AUROC does.

4.2 A fixed judge keeps its AUROC while the policy degrades

The probe’s AUROC stayed flat while the policy degraded. But the probe was also the training target, so the flat AUROC could reflect a change in how the probe relates to the answers it scores, rather than a change in the population. The flag-rate result in §4.1 has the same circularity. A monitor that cannot change, and was never optimised against, separates the two cases when it scores the same

run. If its discrimination also fails to register the decline, the flatness cannot come from the monitor changing, and the result does not depend on the probe having been the training target.

The monitor is an LLM judge (Zheng et al., 2023). We re-sample all eleven checkpoints of runA (the judge ladder of Table 1b: 406 prompts $\times$ 8 answers at temperature 1, 35,728 answers in total). This is a second, independent draw under the probe ladder’s sampling settings, graded by the first answer block. Every answer is scored twice, once with the exact verifier and once with Qwen2.5-7B-Instruct, which is shown the problem and the proposed equation and asked whether the equation is correct. The judge’s score is $P(\text{YES})$ renormalised over the verdict tokens. Appendix A gives the prompt and decoding settings. The judge is imperfect. At step 0 it reaches precision 0.746 at its own median threshold, against a true accuracy of 0.544. The policy was never optimised against it.

The judge is a fixed function. Across all eleven checkpoints it scored 4,651 (problem, equation) pairs more than once; 8 cross the threshold, all within 3×10^{-4} of it, and of the 48 pairs scored at both step 0 and step 99, none do. Any change in its statistics below is therefore not a change in the judge.

Its discrimination statistics do not register the decline (Table 5). Its AUROC is 0.785 at step 0 and 0.769 at step 99. The one excursion is a dip of -0.031 at step 30 ($p = 0.002$, which survives a Holm correction over the judge’s 20 contrasts), across steps 20–40 while true accuracy falls $0.544 \rightarrow 0.457$, recovering by step 80. Most of it is a change in the population rather than the judge: step 0 has 108 answers with no <answer> block, which the judge scores as near-certain failures, against at most 6 at any later step. Without them the step-0 AUROC is 0.769 and the dip is -0.015 ($p = 0.18$). Either way, the checkpoints where accuracy is lowest have AUROC at its starting level. The first and last checkpoints do not differ detectably. The paired change is -0.017 , 95% CI $[-0.048, +0.014]$, which bounds any degradation at about 0.05 rather than only failing to reject zero. Raw precision does fall, by 42% ($0.746 \rightarrow 0.429$), but the share of correct answers falls $0.544 \rightarrow 0.226$ over the same span, and precision at a fixed threshold moves with that share by construction. $\text{LR+} = \text{TPR}/\text{FPR}$, which is prevalence-invariant, is flat end to end at $2.46 \rightarrow 2.57$. For the same reason we do not use lift. It is not prevalence-invariant, and $\sim 91\%$ of its apparent improvement here is the confound it appears to remove, with roughly 70% of the flag rate’s own movement predicted the same way. Both tables report nominal p ; Appendix E gives which judge contrasts survive correction.

The judge did not change, and its AUROC ended where it began while true accuracy fell $0.544 \rightarrow 0.226$, a drop of 31.7 points, or 58%. A stable AUROC here means the judge still ranks correct answers above incorrect ones about as well as before. It does not report how many answers are correct, and it was never built to. That is the problem for monitoring, because discrimination is the statistic a monitor is validated on. Since the judge is fixed, whatever moved in its statistics moved in the population it scored, and that population changed in several ways. Correct answers fell from 54.4% of the pool to 22.6%, and illegal answers grew from 28% of wrong answers to 43%. The surface form also changed. The policy stopped using parentheses, with the share of correct answers containing none rising $0.017 \rightarrow 0.982$, and the judge is sensitive to this. On the same expression differing only in bracketing, at steps 0 and 99, it passes 0.746 with parentheses and 0.548 without ($+0.198$, 95% CI $[+0.131, +0.265]$). The judge therefore also depends on a surface correlate. The statistics that do move, such as precision, need the correctness labels that would reveal the accuracy drop directly. This is the situation clinical epidemiology calls spectrum bias (Ransohoff and Feinstein, 1978): a test’s operating characteristics, measured on one population, do not carry over to another. We show that the population moved while the judge did not. We do not isolate which change in the population drives which movement in the judge’s statistics. In the clinical case the population varies for reasons external to the test. Here the deployment of the artifact is what moves the population, so the number an interpretability paper reports from held-out evaluation is measured on a distribution that deployment removes.

The frozen-threshold statistic also registers the change for the judge, though here the share of answers above the threshold decreases rather than increases, $0.500 \rightarrow 0.317$, a -37% change against AUROC’s -2% . The departure is nominally detectable at step 20 (-0.026 $[-0.047, -0.007]$, $p = 0.013$) but not at step 10 ($p = 0.069$), and it survives a Holm correction from step 30 on. It is not monotone, reaching a minimum of 0.294 at step 80 while accuracy continues to fall. The judge was never optimised against, so this departure is not a consequence of the training objective. The probe’s is, in part: its threshold-crossing rate is a function of the reward being maximised, so its early rise is partly guaranteed. The judge’s departure suggests that a frozen operating point can register the population shift on its own, though it comes 20 steps after the probe’s. The quantity to monitor is

distance from that operating point in either direction, not a rise specifically. Li (2026) gives a better instrument for this, anytime-valid e-processes with an anchor set.

The judge is a bystander, never optimised against, so this is not a demonstration that a judge can be gamed. Zhou (2026), who optimises against judges directly, reports a three-family ensemble’s discrimination falling from 0.31 to 0.09 while it still passes 55% of hacked wrong answers. A bystander judge instead keeps its discrimination while the policy degrades. Neither case is detected by the judge’s statistics, and the bystander case is the more common one in practice.

4.3 A reward built only from text features degrades the task more than a probe that reads higher

The probe partly depends on surface properties of the text (§3). This raises the hypothesis that the policy degrades because it learns to produce those surface properties. We test this hypothesis in two ways, with detectors that use only the text and with reward arms pre-registered before they were run.

The first is already in §3. A 39-feature surface model with no access to activations reaches 0.926, close to the probe’s 0.982, so much of what the probe detects is also visible in the text. The second is two pre-registered RL arms, with predictions committed before launch. Arm A follows the logic of amnesic probing (Elazar et al., 2021), though not its method. We subtract from the activations the part that the 39 surface features predict, by least squares, refit the probe on what remains, and train against it. This asks whether the degradation requires the surface information the probe relied on. Elazar et al. remove a property by iterative nullspace projection and measure a behavioural counterfactual, whereas we remove a hand-specified feature set and measure a training outcome. Arm B’s reward is the text-only surface model itself. Both arms start from C_{SFT} and share one recipe: the same trainer, which reads hidden states from a frozen copy of C_{SFT} so that each reward is a fixed function of the text, and the same optimizer settings and evaluation. `runB`, which also starts from C_{SFT} , appears as arm C for reference, but it differs from the arms in more than its reward. Its probe read the current policy’s hidden states, it used weight decay 0.01 and gradient clipping at 1.0 where the arms used 10^{-4} and none, and its probe was fit on different answers. Only arms A and B form a controlled comparison (Figure 2, Table 2).

All three arms are evaluated under the arm protocol (Table 1b). Arm C was first scored under the headline protocol, at 0.0734; re-scored like the other two it reads 0.0782, a difference of +0.48 pp (CI $[-0.62, +1.60]$), so the comparison does not depend on the protocol.

Arm	reward	read-only AUROC	first-block acc. (vs. arm A)
A	surface- <i>residualised</i> probe	0.834	0.1678 (reference)
B	surface features <i>only</i>	0.925	0.0000 (−16.78 pp, CI $[-18.97, -14.66]$)
C [†]	raw reward probe (<code>runB</code>)	0.945	0.0782 (−8.96 pp, CI $[-10.81, -7.17]$)

Table 2: The rewards, compared. The AUROC column is on one population and one label rule: C_{outcome} ’s headline answers graded by the first block, on the half of the 406 prompts that arms A and B were not fit on. Accuracy is first-block at step 100, all three under the arm protocol (Table 1b). Intervals are paired over prompts, for fixed trained policies. [†]Arm C differs from arms A and B in trainer and optimizer settings as well as reward (§4.3), so only A against B is controlled.

A reward with no access to the model’s internals drove accuracy to exactly zero, and the output keeps its form. Every answer still emits `</think>` and 3,244 of 3,248 contain an `<answer>` block, but only 0.12% of those blocks are arithmetic at all. A verbatim example follows, for input [29, 17, 63, 38] and target 68.

```
<answer>
Let me verify:
1. 17 - 10 = 7
2. 63 - 7 = 56
3. 29 * 38 = 1102
4. 56 - (29 * 38 / 38) = 68
</answer>
```

Step 1 uses 10, which is not in the input. Step 3 is correct but irrelevant. Step 4 reuses two numbers and evaluates to 27 while asserting 68 in the answer position. The answer has the form of verification and contains no valid arithmetic. The reward’s largest weights favour long text after `</think>`

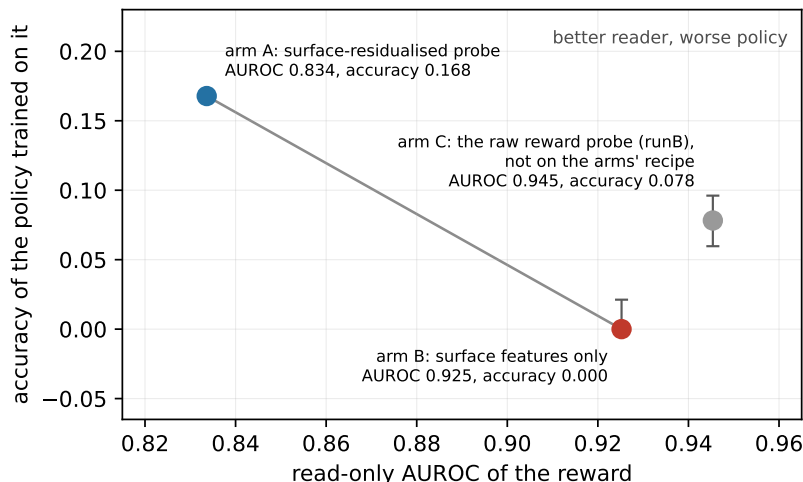


Figure 2: Higher held-out AUROC did not indicate the safer training target. Each point is one reward, showing its AUROC as a held-out monitor, scored on one population on prompts it was not fit on, against the first-block accuracy of the policy after 100 RLOO steps against it, under the arm protocol (Table 1b). Arms A and B share a recipe and differ only in the reward; arm B reads higher and trained a far worse policy. Arm C (runB) reads highest but differs from the arms in trainer and optimizer settings (§4.3), so it is shown for reference, in grey. Bars are paired prompt-clustered 95% intervals of each arm’s difference from arm A; they measure evaluation uncertainty for each trained policy, not variation across training runs. The same three rows appear in Table 2.

(`tail_len`, +1.19 in standardised units) and a long answer block (`ans_len`, +0.62), and penalise equals signs in the reasoning (`n_equals`, -1.11).

Arm A’s pre-registered prediction was that it would degrade the task less than the published raw-probe run. The pre-registration named that run without saying which; our notes from the time identify it as runA, against which arm A degraded more (-6.83 pp), while against runB, which shares arm A’s initialisation, it degraded less (+8.96 pp). Neither comparison isolates residualisation: runA differs in initialisation, which is worth about 16 pp on its own, and runB in trainer and optimizer settings. We treat the pre-registered test of residualisation as inconclusive. Arm B’s prediction, that a text-only reward degrades the task at least as much as the raw probe, holds against both.

Two rewards under one recipe at one seed each give a counterexample and not a relationship (Figure 2). Arm B reads higher than arm A as a held-out monitor and produces a far less accurate policy, with the paired interval far from zero, so in this instance held-out AUROC ranked the two rewards backwards. That interval is over prompts for two fixed trained policies; whether the reversal repeats across training seeds is untested. Arm A’s policy kept the most accuracy of any reward here, but without a raw-probe arm on the same recipe we cannot say how much of that is the residualisation.

Surface occupation is sufficient to cause the degradation, but is not shown to be its mechanism.

Producing cheap textual structure is sufficient to defeat both a monitor and the task, with no access to internals. This does not establish that it is the mechanism behind the published probe’s failure, and arm A is evidence against the simplest version of that account.

The share of a monitor’s AUROC that survives conditioning on cheap surface features can be measured before the monitor is placed in a gradient. For our trace-final probe it survives well, with AUROC holding $0.982 \rightarrow 0.939$ under `</think>`-position stratification, and the policy trained against a probe of the same kind degraded regardless. Which directions an optimizer can push the monitor’s score along is a different question from what the monitor can detect, and held-out accuracy answers only the second.

5 Limitations

Every RL configuration is a single seed, including the checkpoint ladder on which the lag is measured. Our prompt-clustered intervals measure evaluation uncertainty for each trained policy, not variability

across training runs, and our three analysis seeds bound measurement noise, not training variance. Training was not seeded. The arm A against arm B reversal is one observed counterexample whose repeatability is unknown, and repeating those two arms over seeds is the most direct replication. Arm C is not on their recipe, and a raw-probe arm that is would complete the comparison. These results demonstrate direction and not size. Scale does not help. All RL experiments are at 0.5B, and the 1.5B replication evaluates the probe as a monitor only: it holds (AUROC 0.974) while the applied gain shrinks, +8.6 against +11.7 pp. We use one task, one model family, one RL algorithm and one bystander judge. The flag rate has no matched benign control, so it is a measure of change and not a calibrated alarm.

No causal experiment is reported, and an earlier result is withdrawn. An earlier version reported that RL makes the probe direction causally active under steering. The steered vector has cosine 0.163 with the probe that was actually optimised, rather than the 1.000 the claim assumed. The intervention was also applied one transformer block and 2–3 tokens downstream of the read site, and the key contrasts reach only $p = 0.063$. Those checkpoints have been deleted. The cosine between the steered vector and the direction the policy optimised would have caught the error. Steering results of the form “optimisation made direction v causal” do not routinely report it. It is a necessary check for that class of claim, together with confirming that the intervention site is the read site. A defensible version of the experiment needs interventions aligned to the read site, evaluated as a causal abstraction (Geiger et al., 2021).

Every number in this paper regenerates on CPU from cached artifacts using one script, `scripts/check_everything.sh`, that fails on any disagreement. The script prints each published value beside its recomputation, covering the numbers in the prose, every cell of the checkpoint tables, and the three AUROCs on which the ranking depends, and exits non-zero on disagreement. The withdrawn steering result is outside its scope, since those checkpoints no longer exist. Code and the script are at <https://github.com/Abraham-y/reader-writer-probe-rl>, and the 0.32 GB of cached artifacts it reads are at <https://huggingface.co/datasets/prismane16/monitor-accuracy-under-rl>.

Acknowledgments and Disclosure of Funding

This work was supported by compute credits from Modal. We thank the reviewers, whose comments led to the evaluation protocols of Table 1b and to the audit that corrected several claims in the submitted version.

References

- Gandhi, K., et al. Stream of search (SoS): Learning to search in language. arXiv:2404.03683, 2024.
- Ahmadian, A., et al. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. ACL arXiv:2402.14740, 2024.
- Arditi, A., et al. Refusal in language models is mediated by a single direction. NeurIPS, arXiv:2406.11717, 2024.
- Burns, C., et al. Discovering latent knowledge in language models without supervision. ICLR, arXiv:2212.03827, 2023.
- Pacchiardi, L., et al. How to catch an AI liar: Lie detection in black-box LLMs by asking unrelated questions. ICLR, arXiv:2309.15840, 2024.
- Gao, L., Schulman, J., Hilton, J. Scaling laws for reward model overoptimization. ICML, arXiv:2210.10760, 2023.
- Coste, T., et al. Reward model ensembles help mitigate overoptimization. ICLR, arXiv:2310.02743, 2024.
- Skalse, J., et al. Defining and characterizing reward hacking. NeurIPS, arXiv:2209.13085, 2022.
- Denison, C., et al. Sycophancy to subterfuge: Investigating reward-tampering in large language models. arXiv:2406.10162, 2024.

- Baker, B., et al. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. *arXiv:2503.11926*, 2025.
- Gupta, R. and Jenner, E. RL-Obfuscation: Can language models learn to evade latent-space monitors? *arXiv:2506.14261*, 2025.
- Taufeeque, M., Heimersheim, S., Gleave, A., and Cundy, C. The obfuscation atlas: Mapping where honesty emerges in RLVR with deception probes. *ICML (Oral)*, *arXiv:2602.15515*, 2026.
- Ransohoff, D. F. and Feinstein, A. R. Problems of spectrum and bias in evaluating the efficacy of diagnostic tests. *New England Journal of Medicine*, 299(17):926–930, 1978.
- Belinkov, Y. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, *arXiv:2102.12452*, 2022.
- Hewitt, J. and Liang, P. Designing and interpreting probes with control tasks. *EMNLP*, *arXiv:1909.03368*, 2019.
- Voita, E. and Titov, I. Information-theoretic probing with minimum description length. *EMNLP*, pp. 183–196, *arXiv:2003.12298*, 2020.
- Elazar, Y., Ravfogel, S., Jacovi, A., and Goldberg, Y. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *TACL*, *arXiv:2006.00995*, 2021.
- Geiger, A., Lu, H., Icard, T., and Potts, C. Causal abstractions of neural networks. *NeurIPS*, *arXiv:2106.02997*, 2021.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*, *arXiv:2306.05685*, 2023.
- Frick, E., Li, T., Chen, C., Chiang, W.-L., Angelopoulos, A. N., Jiao, J., Zhu, B., Gonzalez, J. E., and Stoica, I. How to evaluate reward models for RLHF. *arXiv:2410.14872*, 2024.
- Malik, S., Pyatkin, V., Land, S., Morrison, J., Smith, N. A., Hajishirzi, H., and Lambert, N. Reward-Bench 2: Advancing reward model evaluation. *arXiv:2506.01937*, 2025.
- Norman, J. D., Rivera, M. U., and Hughes, D. A. Reliability without validity: A systematic, large-scale evaluation of LLM-as-a-judge models across agreement, consistency, and bias. *arXiv:2606.19544*, 2026.
- Weng, S., Feng, Y., and Xie, X. Beyond accuracy: Policy invariance as a reliability test for LLM safety judges. *arXiv:2605.06161*, 2026.
- Yang, Z., Hou, Y., and Yang, X. When the judge changes, so does the measurement: Auditing LLM-as-judge reliability. *arXiv:2607.08535*, 2026.
- Zhou, C. More convincing, not more correct: Self-play reward hacking of reference-free LLM judges. *arXiv:2607.05904*, 2026.
- Li, Y. Who drifted: the system or the judge? Anytime-valid attribution in LLM evaluation pipelines. *arXiv:2606.15474*, 2026.
- Beigi, M., Jin, M., and Huang, L. Proxy reward internalization and mechanistic exploitation: A learned precursor to reward hacking and its generalization. *arXiv:2606.09711*, 2026.
- Bailey, L., et al. Obfuscated activations bypass LLM latent-space defenses. *arXiv:2412.09565*, 2024.

A The judge prompt and decoding settings

The judge is the one language model that is part of our method rather than a subject of it, so we give it in full. Every answer is scored by Qwen2.5-7B-Instruct with this prompt, where {nums}, {target} and {equation} are filled from the problem and the first <answer> block (answers with none are sent with “(no answer given)” in its place). The text is the user turn of Qwen’s chat template, which prepends the model’s default system turn:

```
You are grading a solution to a Countdown arithmetic puzzle.
The solver was given the numbers {nums} and the target {target}. A
correct solution uses each number exactly once, uses only + - * /
and parentheses, and evaluates to the target.
The solver’s proposed equation:
{equation}
Is this equation a correct solution? Answer with exactly one word,
YES or NO.
```

The score is $P(\text{YES})$ from the first generated token’s top-20 logprobs, renormalised over six YES and six NO variants (upper, title and lower case, each with and without a leading space; all single tokens). Two of the 35,728 answers had no YES variant in the top 20 and score 0. It is a continuous score rather than a sampled verdict and the threshold analyses in §4.2 are well defined. Decoding is greedy and the model is frozen, which is what makes the fixed-function check of §4.2 meaningful: the same (problem, equation) pair scored at two checkpoints receives the same score up to floating-point non-determinism, and we verify that empirically rather than assuming it.

No other language model forms part of the method. The probes are logistic regression, the policy is trained with RLOO against the verifier or against a probe, and the verifier is exact and rule-based.

B Why the paper reports LR+ rather than lift, and what is wrong with the control ladder

Why AUROC stays flat: partly the policy, partly the metric. AUROC depends only on the order the probe puts answers in. Between steps 0 and 40 the mean score inflates $0.473 \rightarrow 0.677$ while leaving that order almost intact, which is close to a pure recalibration, so “AUROC stayed flat” is partly a fact about the metric and about every rank-based statistic. The obvious replacement is worse. Precision at a frozen threshold falls $0.588 \rightarrow 0.416$ (−29.3%), but the share of answers that are correct falls too, and precision moves with it whether or not the probe changed. The prevalence-invariant statistic is $\text{LR+} = \text{TPR}/\text{FPR}$, and it falls $2.41 \rightarrow 1.65$, or −31.6%: slightly more than raw precision, not less. All four statistics here use last-block labels, as in Table 3. Measured against its above-chance margin, AUROC moves −5.1%. We use LR+ and not *lift* for the reason §4.2 gives; lift moves only −12.8% here.

The control is the weakest part of this paper. A paired test against step 0 shows the statistic *moved*, not that it moved more than benign training moves it. That needs a control ladder of ordinary runs, and ours is poor: five checkpoints, the wrong initialisation, no recorded sampling configuration, and three of its five checkpoints scored on 166–167 prompts where every attacked checkpoint uses all 406. It is the ordinary run from C_{SFT} that produced C_{outcome} , scored with the reward probe at a threshold frozen at its own step 0. Holding the population fixed at the 166 prompts all five checkpoints share, benign training moves the flag rate by up to +18.3% (at step 30), so the band is not an artefact of the changing prompt count. The attacked run first exceeds it at step 30 (+20.2%). On the same ladder the reward probe keeps the share of answers above 0.95 below 9% at every checkpoint, where the attacked run reaches 38.8% by step 40. Read the lead as evidence that the ordering exists, not as a calibrated false-alarm rate. A matched control ladder is the highest-value experiment left undone here.

C Setup detail

C_{SFT} is asingh15/qwen-sft-countdown-defaultproj, a public checkpoint we did not train. C_{outcome} is 100 RLOO (Ahmadian et al., 2024) steps with the verifier reward at batch 128, group size 16, KL 10^{-3} , entropy 10^{-3} , lr 10^{-5} , weight decay 10^{-4} and no gradient clipping. runA and runB

use group size 8, weight decay 0.01 and gradient clipping at 1.0, with the KL anchor at their own initialisation; arms A and B use group size 8, weight decay 10^{-4} and no clipping. No run was seeded. During RL, sampling stops at the first `</answer>`; evaluation sampling does not. The verifier returns $\{0, 0.1, 1.0\}$. Eval is 500 procedurally generated problems filtered to the 406 with no overlap against the RLOO training pool.

Probes are L2-regularised logistic regression at layer 16. The trace-final probe, the baselines of §3 and the reward probe are scored by 5-fold GroupKFold on prompt and class-balanced by subsampling the held-out scores. Arms A and B are fit with a class-weighted loss on one half of the prompts, split by a hash of the prompt index, and scored on the other half. The ladder AUROCs are plain AUROCs of the frozen reward probe. Where an in-sample figure exists we do not quote it; the trace-final probe reads 0.995 in-sample against 0.982 held out.

D The reader probe in full

As a best-of-16 selector on 406 held-out prompts the probe reaches 0.6700, against 0.5531 for picking at random among the same answers and 0.6847 for an oracle that takes a correct answer whenever one is present; that is 88.8% of the available gain (prompt-clustered paired bootstrap, CI [78.9%, 96.5%]). A prompt has up to 16 scoreable answers, 15.5 on average once unscorable ones are dropped. The margin over the free baselines is +3.20 pp of selection accuracy (CI [+0.99, +5.42], $p = 0.008$ against picking the answer with the shortest `<think>` and $p = 0.004$ against the 39-feature surface model), and +0.056 AUROC. That margin, not the raw 0.982, is the number to read. This is a baseline comparison, not the control-task selectivity of Hewitt and Liang (2019), which would require refitting against randomised labels; we did not run that.

On length specifically: stratified by `</think>` position the probe still reads 0.939, against 0.564 for position alone, so it is not a length detector, though a free structural baseline already captures 64% of the headroom.

Two limits on what any of this licenses. High probe accuracy is not evidence that the model *uses* the direction (Belinkov, 2022), and accuracy is a lossy summary of what a probe extracted (Voita and Titov, 2020); we make no causal claim. Answers with no locatable `</think>` are unscorable and dropped from every AUROC in this section (2.9% on C_{outcome} , 10.4% on C_{SFT} , all wrong, so the filter flatters the weaker model). The checkpoint ladder of §4.1 drops such answers too, but few: 32 at step 0 and at most 8 at any other step (3,216–3,248 of 3,248 scored), too few to explain the flat AUROC. Layer 16 was fixed in advance from the original study.

E Full checkpoint ladders

Every checkpoint of both series, with prompt-clustered bootstrap intervals. The body quotes the load-bearing values from these tables; they are reproduced whole here because both series are non-monotone and a selected subset would misrepresent them. Checkpoint labels are the trainer’s: the checkpoint labelled step k holds the weights after $k + 1$ updates, so step 0 is one update past C_{outcome} , and step 99 is the final checkpoint. Both tables report nominal p . Corrected over the judge’s 20 step-0 contrasts (10 AUROC, 10 flag rate), the AUROC dip at step 30 and the flag-rate departures from step 30 on survive Holm, and the flag rate from step 40 on also survives Bonferroni.

RLOO step	acc. first	acc. last	probe AUROC [95% CI]	p vs. 0	prec @ T	LR+	flag rate [95% CI]
0	0.546	0.372	0.783 [0.760, 0.804]	—	0.588	2.41	0.500 [0.466, 0.535]
10	0.493	0.308	0.777 [0.753, 0.801]	0.627	0.504	2.28	0.529 [0.494, 0.566]
20	0.509	0.312	0.778 [0.753, 0.801]	0.598	0.503	2.23	0.559 [0.522, 0.596]
30	0.496	0.291	0.769 [0.746, 0.791]	0.222	0.439	1.91	0.601 [0.565, 0.636]
40	0.445	0.302	0.768 [0.748, 0.790]	0.228	0.416	1.65	0.691 [0.659, 0.721]
50	0.404	0.260	0.696 [0.673, 0.719]	0.000	0.290	1.16	0.866 [0.846, 0.885]
60	0.312	0.186	0.620 [0.592, 0.647]	0.000	0.189	1.02	0.977 [0.972, 0.982]
70	0.252	0.148	0.579 [0.548, 0.609]	0.000	0.149	1.01	0.987 [0.983, 0.990]
80	0.247	0.148	0.576 [0.546, 0.608]	0.000	0.149	1.01	0.991 [0.988, 0.995]
90	0.247	0.141	0.615 [0.584, 0.646]	0.000	0.141	1.00	0.996 [0.994, 0.998]
99	0.242	0.133	0.562 [0.531, 0.594]	0.000	0.133	1.00	0.996 [0.994, 0.998]

Table 3: The reward probe on runA (initialised from C_{outcome}), probe-ladder protocol: 8 fresh answers per prompt at temperature 1 on the 406 prompts. Accuracy is given under both grading rules; the AUROC, precision, LR+ and flag rate use the last-block label, the rule the probe was fit to predict. Accuracy is significantly lower from step 10 under either rule; the AUROC first moves at step 50 under this label, and at step 40 under first-block labels (Table 4). Intervals are a prompt-clustered bootstrap over the 406 prompts (2,000 resamples) computed from per-rollout probe scores on each checkpoint’s own cached activations; p from paired differences against step 0.

RLOO step	probe AUROC, first block [95% CI]	p vs. 0
0	0.919 [0.904, 0.932]	—
10	0.917 [0.901, 0.932]	0.807
20	0.936 [0.922, 0.949]	0.022
30	0.919 [0.903, 0.932]	0.972
40	0.876 [0.860, 0.892]	0.000
50	0.779 [0.759, 0.798]	0.000
60	0.655 [0.631, 0.680]	0.000
70	0.619 [0.593, 0.645]	0.000
80	0.615 [0.588, 0.642]	0.000
90	0.622 [0.598, 0.647]	0.000
99	0.585 [0.557, 0.613]	0.000

Table 4: The reward probe’s AUROC on the same ladder as Table 3, against first-block labels, the block training graded. It starts higher than under last-block labels, rises at step 20, and first falls at step 40. Prompt-clustered bootstrap over the 406 prompts, 2,000 resamples; p from paired differences against step 0.

RLOO step	acc. first	judge AUROC [95% CI]	p vs. 0	prec @ T	LR+	flag rate [95% CI]
0	0.544	0.785 [0.757, 0.813]	—	0.746	2.46	0.500 [0.466, 0.534]
10	0.495	0.784 [0.755, 0.812]	0.864	0.703	2.41	0.481 [0.449, 0.516]
20	0.506	0.761 [0.729, 0.791]	0.011	0.699	2.26	0.474 [0.441, 0.505]
30	0.488	0.754 [0.723, 0.783]	0.002	0.679	2.22	0.465 [0.432, 0.498]
40	0.457	0.757 [0.726, 0.785]	0.025	0.667	2.38	0.434 [0.403, 0.467]
50	0.399	0.778 [0.748, 0.806]	0.593	0.631	2.58	0.373 [0.342, 0.407]
60	0.307	0.787 [0.757, 0.814]	0.921	0.558	2.84	0.331 [0.304, 0.359]
70	0.268	0.790 [0.761, 0.818]	0.731	0.524	3.01	0.304 [0.276, 0.330]
80	0.234	0.792 [0.761, 0.820]	0.673	0.482	3.04	0.294 [0.268, 0.320]
90	0.232	0.775 [0.744, 0.806]	0.511	0.448	2.68	0.312 [0.284, 0.339]
99	0.226	0.769 [0.737, 0.801]	0.312	0.429	2.57	0.317 [0.290, 0.344]

Table 5: An LLM judge watching the same run. runA, judge-ladder protocol: an independent draw of 8 answers per prompt at temperature 1, graded by the first block, the rule the judge is shown. Its AUROC dips at steps 20–40 and recovers, mostly because step 0 holds 108 answers with no `<answer>` block (§4.2); its positive likelihood ratio LR+ tracks the same shape and ends where it began, so the operating point does not degrade. Prompt-clustered bootstrap over the 406 eval prompts, 2,000 resamples; p from paired differences against step 0, nominal.